What a Cheap Verifier Can and Cannot See: Telemetry Monitoring, Deterministic Checks, and Repair for Tool-Using LLM Agents

Anonymous Author(s)
Affiliation
Address
email

Abstract

Verifying an agent after it answers is too late, and verifying every step with a second model costs more than the agent. We ask what is verifiable from *observable step telemetry alone* — semantic embeddings of step output, token uncertainty, action metadata — by monitors that cost microseconds and are fitted on healthy runs only, and where that cheap signal provably runs out. On 2,823 committed episodes across three frameworks, four agent models and a commercial API, a one-class echo-state-network CUSUM ensemble detects 0.71 ± 0.07 of failures at a 5% false-alarm budget. Two findings bound it. First, a *horizon law*: the temporal monitor's advantage over a memoryless baseline grows from $+0.09$ detection points at ≤ 3 post-onset steps to $+0.40$ at ≥ 9, and this law predicts its own failure region on two external corpora we did not build — on AFTraj-2K, where 94% of failures end within eight steps of onset, pooled detection is 0.048 exactly as the law forecasts, while at ≥ 9 steps it is 0.509. Second, an *observability boundary*: a wrong-but-well-formed value perturbs no behavioural channel, so no reference-free monitor can see it. We therefore pair the monitor with deterministic checks that need no null and no threshold — recomputing a run's stated total from the tool results it actually received — which catch 60% of failures at **0 of 63** false positives against the monitor's 54% at 17%, transfer across model families unchanged (110/110 at 0/10), and close into repair that lifts task success from 52% to 73%. The contribution is a measured decomposition of which cheap observable verifies which failure, and an explicit statement of what neither verifies.

1 Introduction

An agent episode is a sequence of steps, each emitting observable telemetry: what the agent said, how confident its tokens were, which tool it called, what came back, how long it took. The verification question is whether that stream is enough to decide, *mid-run*, that something has gone wrong — early enough to halt, roll back, or repair.

Two answers dominate. Check the final output, which is cheap but arrives after the budget is spent; or judge every step with a second LLM, which is accurate and costs a model call per step. This paper measures a third position: a one-class monitor over step telemetry at $\sim$200 μs per step, fitted on healthy runs only, with no failure labels and no second model. We are interested less in its headline accuracy than in its *boundary* — what such a verifier structurally cannot see, and what to pair with it there.

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

Contributions.

1. **A horizon law with external validation.** The temporal monitor's advantage over a memoryless baseline is governed by post-onset runway, not by detector identity. The law predicts where the monitor will fail, and holds on two corpora built by other groups (§6).
2. **A measured observability boundary.** Content corruption that changes data without changing behaviour is invisible to behavioural telemetry; we show richer embeddings do not fix it, and that a deterministic grounding channel does (§8).
3. **Deterministic verification that needs no null.** Checks that recompute a run's answer from the tool results it received beat the monitor on precision by construction — 0% false positives against 17% — and transfer across model families with nothing retuned (§9).
4. **Verification closed into repair**, with the negative results kept: rolling a flagged run back and re-running it lifts task success from 52% to 73%, but only one of five failure classes is actually repairable (§9).
5. **A measured LLM judge**, not a stipulated one. The judge everyone escalates to scores $p_{\text{detect}} = 0.548$ where analyses assume 0.90; substituting the measured value drops our own escalation claim from 83% to 44% recovery, and we report the lower number (§7).

2 Telemetry and the Monitoring Problem

An episode is a sequence $\mathcal{E} = \{(q_t, a_t, o_t, r_t)\}_{t=1}^{T}$ of reasoning q_t, action a_t, observation o_t and runtime metadata r_t. At each step the adapter emits a causal feature vector

$$\mathbf{x}_t = [\,\mathbf{e}_t;\ \mathbf{u}_t;\ \mathbf{m}_t;\ \mathbf{g}_t\,] \in \mathbb{R}^d, \tag{1}$$

using only information available at or before t: $\mathbf{e}_t$ a 32-dim deterministic character-trigram hash of the step text (no model, no network), $\mathbf{u}_t$ four token-surprisal aggregates where the provider exposes logprobs, $\mathbf{m}_t$ action type, latency, output length, retry and error metadata, and $\mathbf{g}_t$ nine content-grounding features (§8). Monitors are fitted on healthy episodes only and standardised with healthy statistics, $\tilde{\mathbf{x}}_t = \mathbf{D}^{-1}(\mathbf{x}_t - \boldsymbol{\mu})$.

For a failed episode with onset τ, a detector alarms at $\hat{\tau} = \inf\{t : s_t > \gamma\}$. We report detection at a *realized* false-alarm rate rather than AUROC alone: a monitor that ranks well can still alarm after the run ends or interrupt too many healthy runs, and both matter for a verifier that is allowed to halt.

3 Streams and the Serving Decision

No single detector observes every failure mechanism, so the system is a set of independently calibrated streams.

Temporal stream. An ensemble of $K{=}8$ leaky echo-state reservoirs [3] predicts the next standardised step; member k has state $\mathbf{h}_t^{(k)}$ and surprise $e_t^{(k)} = \|\tilde{\mathbf{x}}_t - \hat{\mathbf{x}}_t^{(k)}\|_2$. The ensemble exposes mean surprise and disagreement, which are robustly z-scored and accumulated by a one-sided CUSUM [6],

$$C_t = \max\{0,\ C_{t-1} + z_t - \kappa\}, \tag{2}$$

so that weak but persistent deviations integrate rather than being smoothed away. Everything in (2) depends only on $\mathbf{x}_{0:t}$ and fit-time quantities: strictly causal, no lookahead.

Abrupt-change stream. Accumulation is useless when one or two steps remain after onset. A memoryless Δ-Mahalanobis stream scores healthy step differences $\Delta\mathbf{x}_t$ against a shrinkage covariance [4], and covers exactly the regime the temporal stream cannot (§5).

Calibration and the serving decision. Each stream's threshold is a quantile of per-episode maxima over *held-out* healthy episodes,

$$\gamma_j = Q_{1-\alpha_j}\Big(\big\{\max_t s_{j,t}^{(i)}\ :\ \mathcal{E}_i \in \mathcal{H}_{\text{val}}\big\}\Big), \tag{3}$$

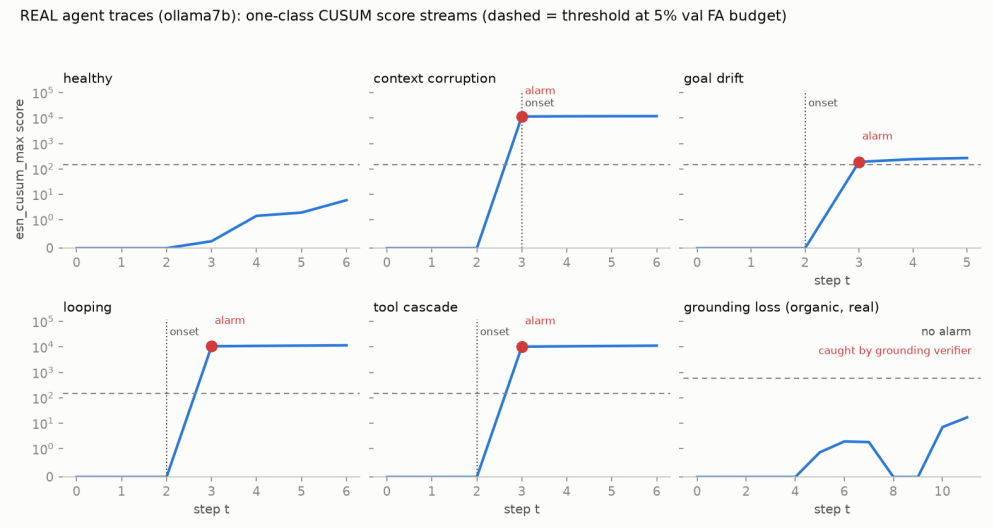

Figure 1: **What the verifier sees.** One-class CUSUM streams per failure class on real agent traces (qwen2.5:7b against real tools); dashed = threshold at the 5% validation budget, dotted = verified onset. Healthy stays two orders of magnitude below the line and every injected class alarms one step after onset. The sixth panel is different in kind: fabrication cannot be injected into a live run, so it is a genuine organic episode — the CUSUM stays flat, because a fabricated figure perturbs no behavioural channel, and the class is caught instead by the deterministic verifier of §9. That is the observability boundary shown rather than asserted. The y axis is symlog: the CUSUM accumulates multiplicatively once a failure takes hold.

and the served alarm is a per-stream OR rather than a fused score,

$$A_t = \bigvee_j \mathbb{I}\big[s_{j,t} > \gamma_j\big] \;\vee\; B_t^{\text{contract}} \;\vee\; B_T^{\text{complete}}. \tag{4}$$

Equation (4) is a design result, not a formality. Under a single shared threshold only episode-max *ordering* matters, so the stream with the heavier healthy tail sets the bar for both: measured, one behavioural healthy episode destroyed **16 of 17** grounding-only detections of context corruption. No monotone rescaling fixes an ordering problem; independent thresholds do, and give a non-degradation property at serving time — no stream's evidence can be numerically cancelled by another's.

4 Real-Ecosystem Validation

Everything claimed about deployment rests on 2,823 committed episodes over 25 corpora — three frameworks, three local models across two families, and the gemini-2.5-flash API — each a live agent run against real tools, with telemetry measured at the adapter rather than constructed. Looping and tool cascade are caught almost everywhere (0.48–1.00) and goal drift wherever it is present (0.66–0.86 on four of five corpora); the weak spots are the content-corruption classes, `wrong_document` (0.00–0.20) and `malformed_json` (0.05–0.50), which is what motivates §8 rather than a rationalisation of it.

Two structural findings shape the rest. First, **monitors do not transfer across deployments**: calibrated on qwen2.5:7b and deployed on llama3.1:8b over a matched task, tool and injector plan, the primary monitor sits at chance (AUROC 0.527, healthy FA 0.75), while the *same* target recalibrated on itself reaches 0.885. That is a property of the calibration, not of a hard corpus, and it means a verifier of this kind is per-deployment by construction. Second, the memoryless baseline wins outright on short-episode datasets — which the next section explains.

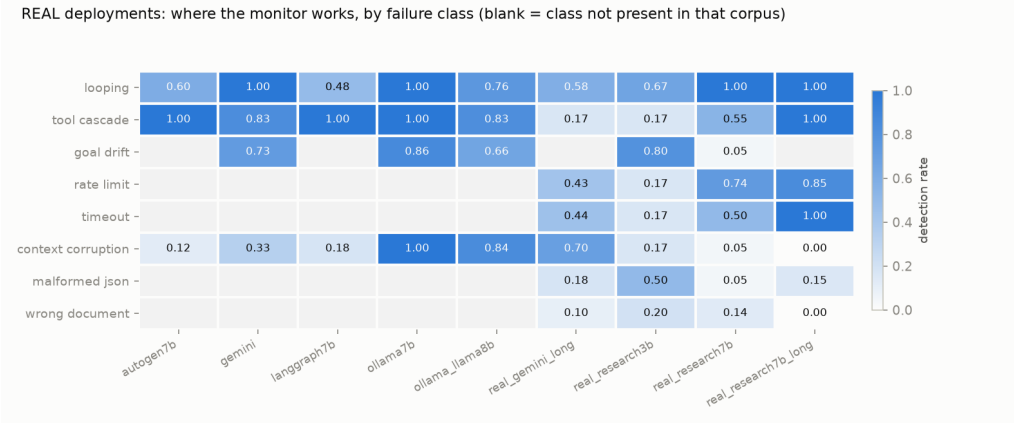

Figure 2: **Where the monitor works, by failure class**, across nine real corpora; blank where a corpus does not contain that class, because absent and undetected are different claims. Looping and tool cascade are caught almost everywhere and goal drift wherever present; the weak spots are exactly the content classes that motivate §8.

5 The Horizon Law

Over 1,002 injected episodes the ESN – Mahalanobis detection gap is $+0.09$ at post-onset horizon ≤ 3 steps, $+0.14$ at 4–8 and $+\mathbf{0.40}$ at ≥ 9 (correlation $+0.25$). The reservoir needs post-onset steps to integrate evidence: where it has them it wins decisively, and where it does not its margin collapses and the memoryless distance becomes competitive. This is a statement about *regime*, and it is what makes the monitor's failures predictable rather than mysterious.

The fusion buys robustness, not dominance. A cross-fit logistic fusion of the two parents reaches grand-mean AUROC 0.826 against 0.802 (ESN) and 0.807 (Δ-Mahalanobis), because each parent collapses on some dataset and the fusion never does. Measured against whichever parent is better *on that dataset*, however, it is at or below it on 7 of 8 datasets in AUROC (mean -0.014) and on all 8 in detection rate (mean -0.140). The hybrid is the right default when the regime is unknown, and the wrong choice when it is known.

6 External Validation on Corpora We Did Not Build

Every number above comes from corpora we collected, so none of them says whether the benchmark is hard. We therefore ran the monitors, unchanged and fitted on each corpus's own healthy runs, against two external benchmarks.

AFTraj-2K [11] contributes 1,882 multi-agent trajectories after the same $T \geq 4$ filter (1,111 healthy, 771 failed) with a judge-annotated earliest-decisive-error step. *Ranking transfers zero-shot*: episode AUROC **0.745** on another group's agents and frameworks, with the uncertainty channel absent entirely, against 0.802 pooled on our own. *The horizon law predicted the operating point, and held*: detection is **0.509** on failures with ≥ 9 steps of runway against 0.170 for the memoryless baseline, but only 53 of 771 AFTraj failures have that much room and 61% end within three steps of onset, so pooled detection is **0.048**. That pooled figure is the mechanism's prediction, not an excuse offered after seeing it: §5 fixed the relationship on our corpora, and AFTraj supplies a failure distribution concentrated where the relationship says detection is impossible. A law that forecasts its own failure region and then survives a corpus it never saw is stronger evidence than a higher pooled number would have been.

ATBench [8] contributes 604 trajectories (279 safe, 325 unsafe) labelled under a different taxonomy of risk source, failure mode and harm. Here the monitor *alarms*: AUROC **0.779** at detection **0.311**, because ATBench's unsafe behaviour is a pattern of action — an over-privileged call, an unvalidated tool result — rather than a single wrong value. Two independent confirmations fall out. The fusion

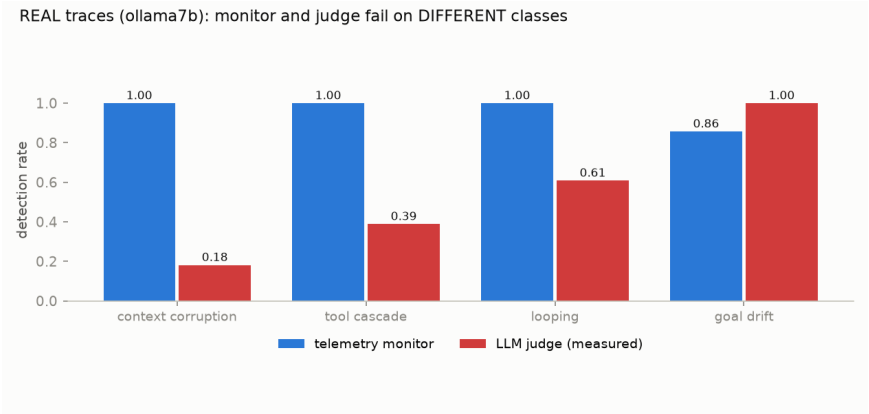

Figure 3: **Monitor and judge fail on different classes**, both measured on the same real corpus. The judge is perfect on goal drift and nearly blind on context corruption; the monitor is the reverse. Complementary verifiers, not substitutes.

collapses to chance (0.457 for the memoryless parent, dragging every 50/50 blend to 0.438–0.463 against the ESN's 0.779 alone), which is §5's pooled-not-dominant finding shown on foreign data. And per failure mode the split is sharp — over-privileged actions 0.508 and unvalidated tool outputs 0.473 against inaccurate information **0.038** — reproducing, on labels we did not write, exactly the content blind spot §8 exists for.

For scale: an LLM auditor on AFTraj reports 66.44 Exact-F1 using a model call at every step [11]. We do not match that and do not claim to; detection at a false-alarm budget is not Exact-F1. The comparison we can make is cost: 173 μs and no model call, against one 7B forward pass.

7 The Judge Is a Verifier Too, and It Should Be Measured

The default answer to "who verifies the agent" is another LLM. Escalation analyses — ours included — usually *stipulate* that judge's operating point. We measured it instead. A gemini-2.5-flash judge on a labelled subset of 161 distinct prompts scores $p_{\text{detect}} = \mathbf{0.548}$ (95% CI 0.44–0.65) and $p_{\text{false}} = \mathbf{0.052}$, against the 0.90/0.02 the analysis assumed; both stipulated values lie outside their measured intervals. Substituting the measured rates and re-running all five master seeds with everything else fixed, the *call saving survives* (6% of judge-every-step calls) but detection recovery falls from 83% to **44%**, supported at **0 of 5 seeds**. We report the escalation layer's detection claim as 44%.

The layer is still worth having, for a reason that only appears once both are measured on the same episodes: judge and monitor fail on *different* classes (Figure 3). The judge is perfect on goal drift (21/21) and nearly blind on context corruption (0.18), where the monitor is the reverse (1.00). That is the empirical case for escalation as a complementary verifier rather than a more expensive approximation of the cheap one — and a caution against citing a stipulated judge as a baseline.

8 The Observability Boundary and the Grounding Channel

Behavioural monitors share one blind spot: corruption that changes *data* without changing *behaviour*. A wrong-but-well-formed number perturbs no channel in (1), so no reference-free detector can reject it. The g channel adds nine causal content features — query–result dissimilarity, result self-consistency, JSON-prefix validity, character anomaly, and a binary lexical-relevance flag costing 2 μs — which makes query–evidence–reasoning *inconsistency* observable without claiming to observe truth. Pooled over 602 injected episodes, the content gate lifts content-class detection $0.23 \rightarrow \mathbf{0.40}$ and behavioural detection $0.73 \rightarrow \mathbf{0.80}$, gaining 37 and 25 discordant detections against zero lost.

The boundary is real and we measured its floor: on a plausible-corruption set, richer MiniLM embeddings leave detection at **0.00** while behavioural AUROC falls 0.82 to 0.70, at 4.4 ms per text and 90 MB. The failure is observability, not feature capacity — which is the argument for §9 rather than for a bigger encoder.

9 Deterministic Verification, and Repair

The monitor needs a healthy null per deployment and spends part of a false-alarm budget. A check needs neither. `total_consistency` recomputes a run's stated total from the tool results *that run actually received*; `required_coverage` confirms every call the task requires was made; `tool_contract` asks whether a result matched any shape its tool can return, and so reports when the result arrives rather than at the answer. None reads the hidden world the task was generated from.

Table 1: Checks against the behavioural monitor, same episodes and same objective labels. Recall is comparable; the difference is precision.

	checks	monitor
failures caught, T=0.2 (served)	60% (**96%** with coverage)	54%
false positives, T=0.2	**0/63 = 0%**	11/63 = 17%
failures caught, T=0.9	65% (**96%** with coverage)	40%
false positives, T=0.9	**0/38 = 0%**	6/38 = 16%

The checks were written by inspecting failures in the serving arm, so that arm cannot also be their test set. A further 120 episodes at disjoint task seeds were collected afterwards and scored with the checks frozen: 54% caught by totals, **93%** with coverage, and **0 of 64** false positives. A llama3.1:8b arm on the *same* task seeds, with nothing retuned, catches **110 of 110** at **0 of 10** — the checks transfer where the monitor does not, because they carry no calibration. Across every labelled corpus `tool_contract` trips on **0 of 1825** healthy episodes and flags 46% of context corruption, **215 of 218** within one step of onset.

Repair. Each flagged episode is rolled back to its last fact-gathering step and re-run, paired on the identical prefix and task; the rollback is real, and success is graded by an oracle the repair prompt never sees. Naming which check failed — without supplying any value — recovers **45%** of genuinely-wrong runs against a 16% resampling control ($p = 0.0005$), which nets task success from **52% to 73%** with zero correct runs broken. Two negative results sit inside that: routing the step to a calculator the agent already holds does *not* beat retry luck ($p = 0.17$), and supplying the recomputed answer buys nothing — 26 of 55 value-bearing hints contain the correct total outright, while the value-free rung recovers at least as much. The recovery does not come from handing over the answer.

Coverage is partial and the shape of the gap is measured: over five injection classes $\times$ five seeds, every behavioural alarm is followed by a repair attempt (21 of 21), but `goal_drift` is the only class a retry fixes (4 of 5). Where the tool layer itself is broken a retry fetches the same broken result, and the value of the intervention is ending the episode fast — a loop trap exits at exactly 10 steps against 30 without the circuit breaker.

10 Limitations

Stated as measured. (1) **No cross-deployment transfer**: the healthy null must be collected under the exact serving distribution; transferred across a model family the monitor sits at chance (0.527) while the same target recalibrated reaches 0.885. (2) **Slow goal drift** evades every per-step surprise monitor tested; the limit is the *rate* of change, not the class — abrupt goal changes are caught at 0.66–0.86. (3) **Content detection is bounded by telemetry completeness**, not by logprob access: ablating the token-surprisal channel moves the ESN by AUROC $+0.000$. (4) **Plausible-value corruption** is undetectable from telemetry by construction. (5) **Fabrication claims are limited by base rate**: 9 of 175 organic episodes are labelled hallucinated and only 2 are fabricated *inputs*, below the pre-registered minimum of 10, so no detection claim is made in either direction; under provocation the grounding verifier catches 0.55 with no healthy false positives — specific, but half sensitive. (6) **Scope**: mock-tool and research-loop tasks, two local model families and one commercial API.

11 Related Work

Concurrent online failure prediction. Detecting failure during an episode is no longer an unoccupied position. AgentForesight reframes attribution as online auditing and releases AFTraj-2K [11]; weakly

supervised early alerting learns turn-level risk from trajectory labels alone [1]. Both use an LLM auditor. We therefore do not claim mid-episode detection from observable telemetry as novel; what is ours is the cost class and the boundary analysis.

Internals versus observables. Hidden-state probes report predicting failure earlier than any observable-behaviour monitor [7]. We do not dispute it: with the weights, activations are richer. The premise here is the deployment where you do not hold them — a hosted API, a third-party agent — and there the choice is between telemetry and nothing.

Runtime enforcement intervenes during execution but requires a specification of what unsafe means: rule sets [9] or labelled unsafe states [10]. Ours is label-free and specification-free, which is why it transfers badly and needs no rule authoring. **Post-hoc attribution** asks which step caused a failure after the fact and finds it hard — 14.2% accuracy on the responsible step [12]; that problem is outcome-conditioned, ours is not. **Hallucination detection** needs model access or repeated sampling [5, 2]; tool-using agents admit the cheaper deterministic route of §9.

12 Conclusion

A cheap verifier over step telemetry covers more of the agent-failure space than its cost suggests, and its failures are predictable rather than arbitrary: the horizon law says where it will not fire, and held on two corpora built by other groups. Where it structurally cannot see — a wrong-but-well-formed value — the answer is not a bigger encoder but a different mechanism, and a deterministic check that recomputes the answer from the evidence the run received is both more precise and more portable than the monitor it supplements. We think the useful framing for agent verification is coverage: which cheap observable verifies which failure, measured, with the gaps stated.

References

[1] Avinash Baidya, Xinran Liang, Ruocheng Guo, Xiang Gao, and Kamalika Das. When evidence is sparse: Weakly supervised early failure alerting in dialogs and LLM-agent trajectories. *arXiv preprint arXiv:2606.05414*, 2026.

[2] Sebastian Farquhar, Jannik Kossen, Lorenz Kuhn, and Yarin Gal. Detecting hallucinations in large language models using semantic entropy. *Nature*, 630:625–630, 2024.

[3] Herbert Jaeger. The "echo state" approach to analysing and training recurrent neural networks. Technical Report 148, GMD – German National Research Institute for Computer Science, 2001.

[4] Kimin Lee, Kibok Lee, Honglak Lee, and Jinwoo Shin. A simple unified framework for detecting out-of-distribution samples and adversarial attacks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.

[5] Potsawee Manakul, Adian Liusie, and Mark J. F. Gales. SelfCheckGPT: Zero-resource black-box hallucination detection for generative large language models. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

[6] E. S. Page. Continuous inspection schemes. *Biometrika*, 41(1/2):100–115, 1954.

[7] Kai Ruan, Zihe Huang, Ziqi Zhou, Qianshan Wei, Xuan Wang, and Hao Sun. Doomed from the start: Early abort of LLM agent episodes via a recall-controlled probe cascade. *arXiv preprint arXiv:2607.06503*, 2026.

[8] Shanghai AI Laboratory. ATBench: A diverse and realistic agent trajectory benchmark for long-horizon agent safety. *arXiv preprint arXiv:2604.02022*, 2026.

[9] Haoyu Wang, Christopher M. Poskitt, and Jun Sun. AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents. In *IEEE/ACM International Conference on Software Engineering (ICSE)*, 2026. arXiv:2503.18666, 2025.

[10] Haoyu Wang, Christopher M. Poskitt, Jiali Wei, and Jun Sun. ProbGuard: Probabilistic runtime monitoring for LLM agent safety. *arXiv preprint arXiv:2508.00500*, 2025.

[11] Boxuan Zhang, Jianing Zhu, Zeru Shi, Dongfang Liu, and Ruixiang Tang. AgentForesight: Online auditing for early failure prediction in multi-agent systems. *arXiv preprint arXiv:2605.08715*, 2026.

[12] Shaokun Zhang, Ming Yin, Jieyu Zhang, Jiale Liu, Zhiguang Han, Jingyang Zhang, Beibin Li, Chi Wang, Huazheng Wang, Yiran Chen, and Qingyun Wu. Which agent causes task failures and when? on automated failure attribution of LLM multi-agent systems. *arXiv preprint arXiv:2505.00212*, 2025.

A Appendix

A.1 Telemetry versions

Version	Dims	Content
v1	43	e_t 32-dim hash embedding; u_t 4 token-surprisal aggregates; m_t action one-hot, log latency, output length, error
v2	43	Tool results appended into step text so corrupted results reach the semantic channel
v3	51	8 derived dims: cosine drift, task-anchor similarity, tool success, retry, per-tool latency, context ratio, reasoning depth, self-consistency
v4	60	9 content-grounding dims: query–result dissimilarity, result self-consistency, JSON-prefix validity, character anomaly, lexical flag

Table 2: Causal telemetry evolution. v4 costs 608 μs/step at the adapter; the lexical flag alone costs 2 μs/result.

A.2 Per-dataset hybrid results

dataset	ESN	Δ-Maha	better parent	logistic	Δ
sim	0.890	0.786	0.890	0.889	-0.000
autogen7b	0.833	0.774	0.833	0.777	-0.056
langgraph7b	0.828	0.885	0.885	0.884	-0.002
ollama7b	0.994	0.895	0.994	0.982	-0.013
real_research3b	0.556	0.665	0.665	0.643	-0.022
real_research7b	0.777	0.848	0.848	0.847	-0.001
real_research7b_long	0.790	0.849	0.849	0.857	$+0.008$
gemini	0.749	0.756	0.756	0.731	-0.025
grand mean	0.802	0.807	0.840	0.826	-0.014

Table 3: Episode AUROC. The fusion beats either parent's grand mean while sitting below the per-dataset better parent on 7 of 8.

A.3 External corpora

corpus	monitor	AUROC	detection	FA
AFTraj-2K	esn_cusum_max	0.745	0.048	0.023
AFTraj-2K	hybrid_weighted50	0.760	0.047	0.018
ATBench	esn_cusum_max	0.779	0.311	0.071
ATBench	Δ-Mahalanobis	0.457	0.268	0.161
ATBench	hybrid_weighted50	0.463	0.277	0.179

Table 4: Monitors run unchanged on two external corpora, fitted on each corpus's own healthy runs. Neither carries token logprobs, so both run on $e{+}m{+}x$ only.

A.4 Reproducibility

Code, all 2,823 committed traces, every result table and the figures are publicly released under a permissive licence; the URL is withheld for review. Each headline number in this paper is tied to a named artifact by a claim ledger that recomputes it from the committed file on every run, and a SHA-256 manifest covers every source file, trace and result so a reader can verify the number in the text came from the file in the repository. An end-to-end behavioural snapshot re-runs the study at a fixed seed and compares every value against a stored baseline.